

# Project 4: Trace-Driven Simulation of Page Replacement (Deadline: 04/30)

This assignment should be programmed entirely in C and must compile and run on the lab's Linux environment or similar. Copying & pasting code from anywhere is considered cheating and results in an automatic F.

## Objectives

Your goal for this assignment is to implement page replacement algorithms used by operating systems.

## Background

If total memory requirements exceed the physical (main) memory, it is necessary to replace pages from memory to free frames for new pages. Two replacement algorithms are First In First Out (FIFO) and Least Recently Used (LRU). Refer to the lecture notes for more information about the two algorithms.

## Programming Task

Write a program (in C/C++) that implements the FIFO and LRU page-replacement algorithms. Use the page reference sequence in the provided input file "pageref.txt" as the input file. Report the number of page faults and latency incurred by each algorithm (see details below). Implement the replacement algorithms so that the number of available page frames can vary as specified (e.g., 20). Assume that demand paging is used, i.e., page frames are initially free.

Requirements:

First, the program should accept three input parameters to specify:

1. The algorithm ("FIFO" or "LRU")
2. The cache size (the number of physical page frames for allocation)
3. The input file that contains the page-reference string.

Example: `$ ./simulate FIFO 20 pageref.txt`

Note: A sample input file is available on Canvas. Your program should be tested using the supplied input file. The format of the input file: Each line has two fields. The first field is either "R" or "W", meaning a read or a write to the page, respectively. The second field is the virtual page number. Thus, each line of the input file gives a reference to a page, either read or write.

Second, The program should generate the following output information:

1. The total number of page references.
2. The total number of page misses.
3. The total number of time units for page misses.
4. The total number of time units for writing back the dirty (modified) page.

Third, the program should calculate the “cost” of page faults in terms of time units. In particular, upon a page hit (the page is found in memory), the reference cost is 0; upon a page miss (the page is not found in memory), it takes 5 time units to load the page in. When deciding to evict a victim page, if the victim page is modified (i.e., the page has been written in memory), it takes 10 time units to write the victim page out.

## Bonus Tasks

1. Implement MIN (Optimal) page replacement algorithm (10%)
2. Implement Clock algorithm (10%)

## Deliverables

- You must create a Makefile to compile your source. (2 pts)
- Your submission should include a README file to clearly explain how to compile and run your code, such as the commands, arguments, and expected input and output, and any necessary information.
- Please submit in the Canvas your source code before 11:59pm on the due date.
- Please include all files necessary for a successful build as a zip file.
- The name of your submission should be *cs460-project4-TeamID.zip*.

## Grading

Your submission will be graded by compiling and running it.

- Please make sure your source code can compile. Absolutely no credit if it does not compile.
- Please make sure your programs run on servers in ENCS computer labs.
- Please don't include the binary files. Do a make clean before submission.
- Please don't leave out any files!

- Please don't modify any files you don't need to!
- Please don't send us the meta-information from your revision control system!